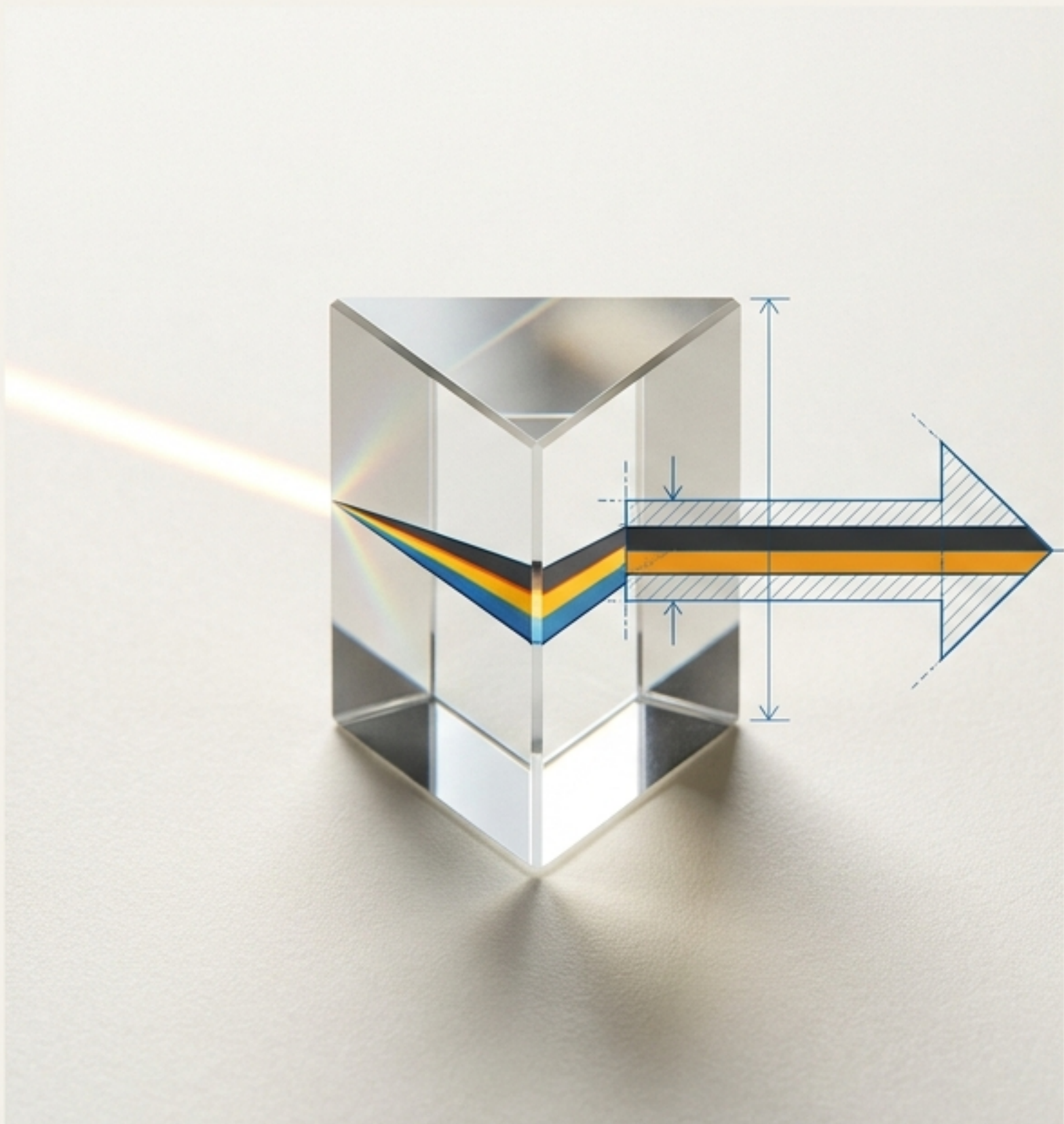


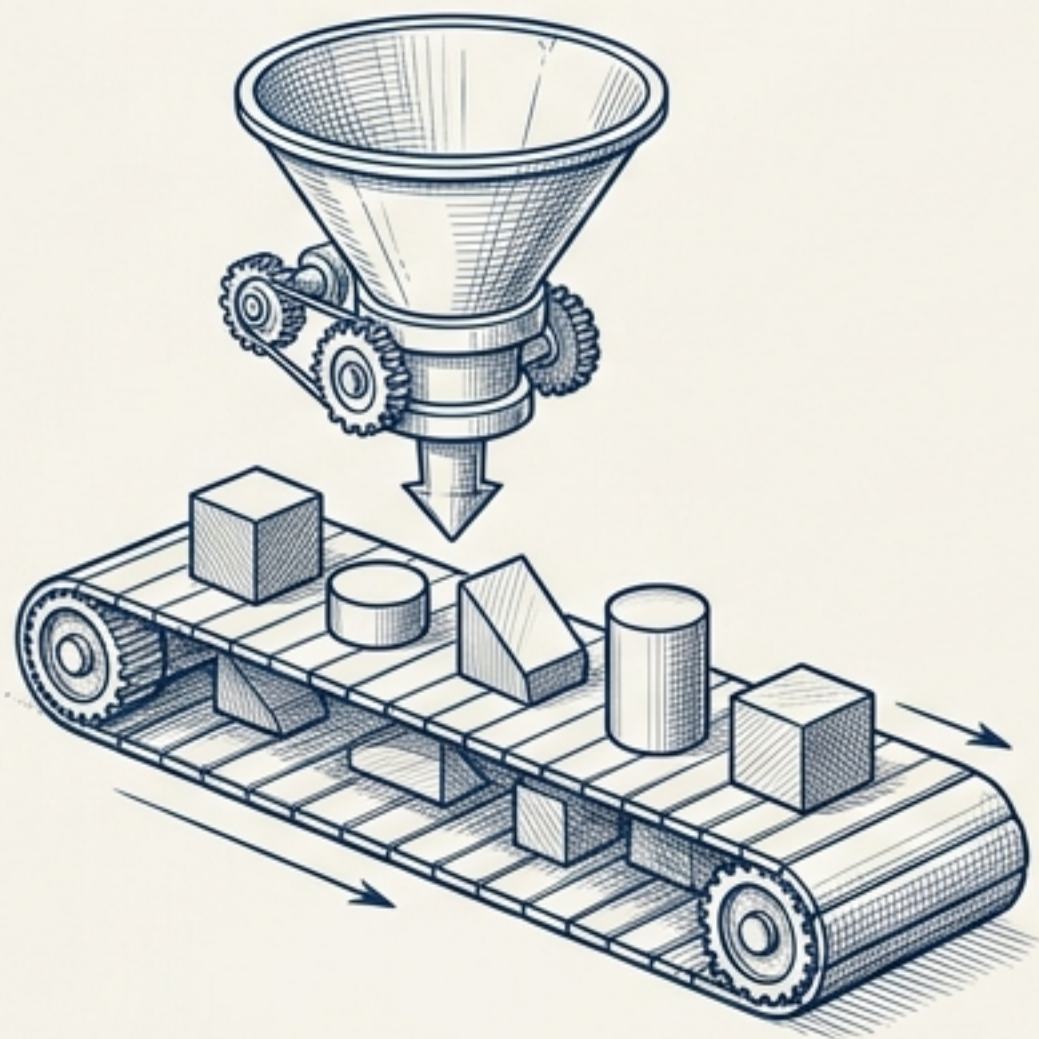


AI 時代最稀缺的能力：產品感

寫程式變便宜，判斷力變昂貴。
給軟體工程師與PM的新時代決策指南。

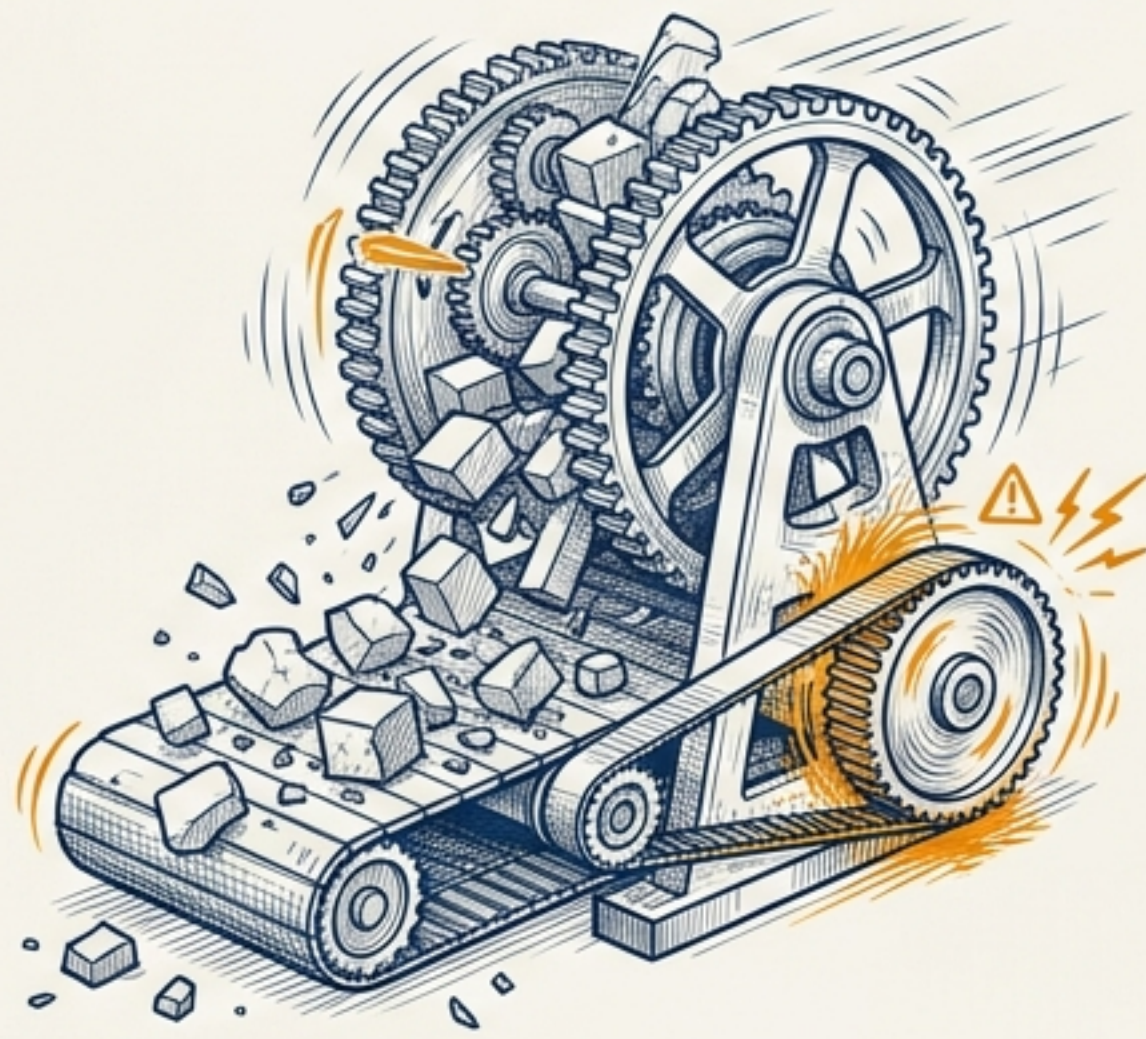
AI 帶來的生產力陷阱：做得快，不等於做得對

過去（開發慢）



需求自然被「時間成本」過濾。
做不完的，通常是最不重要的。

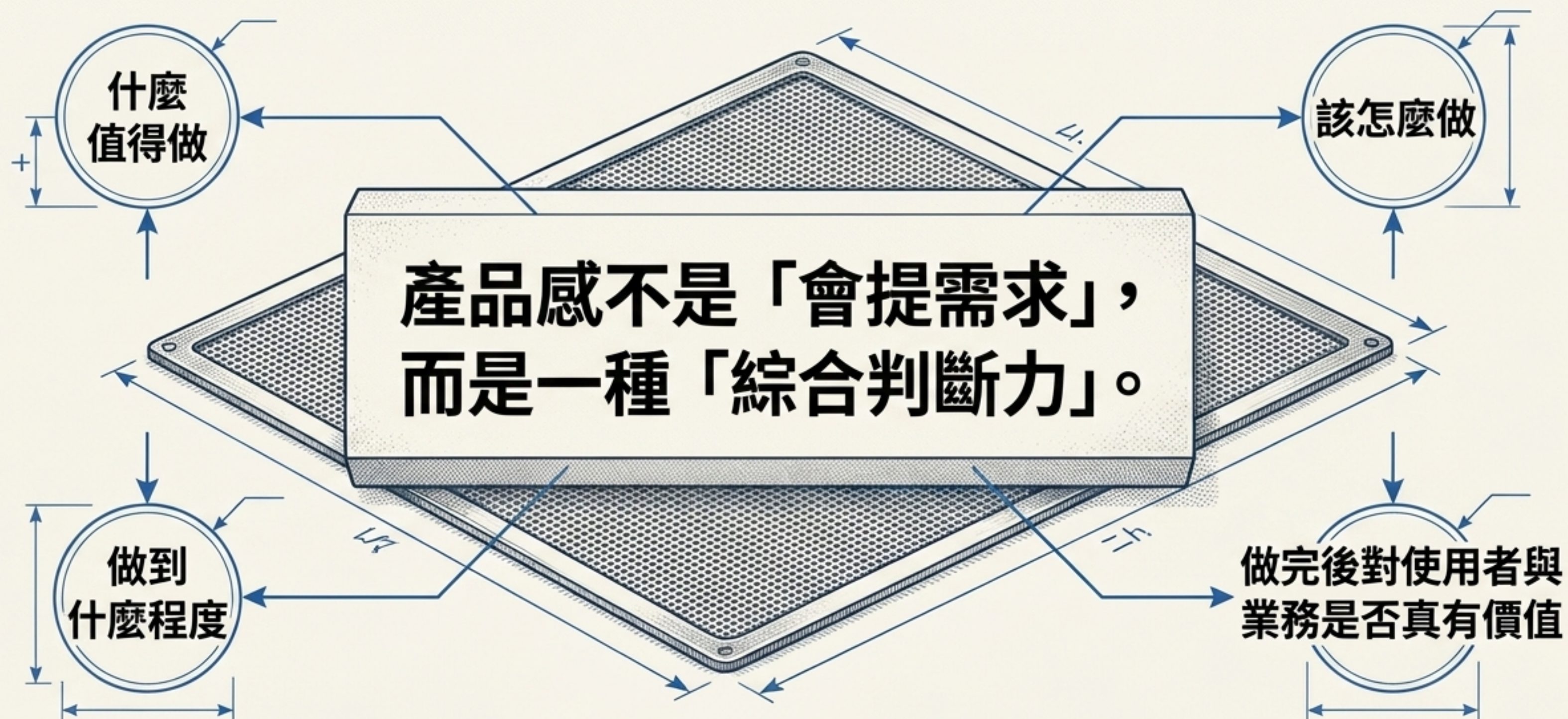
現在（AI 讓開發變快）



輕易做出大量低價值功能。方向變分散，
團隊忙得很快，但價值卻沒有等比例增加。

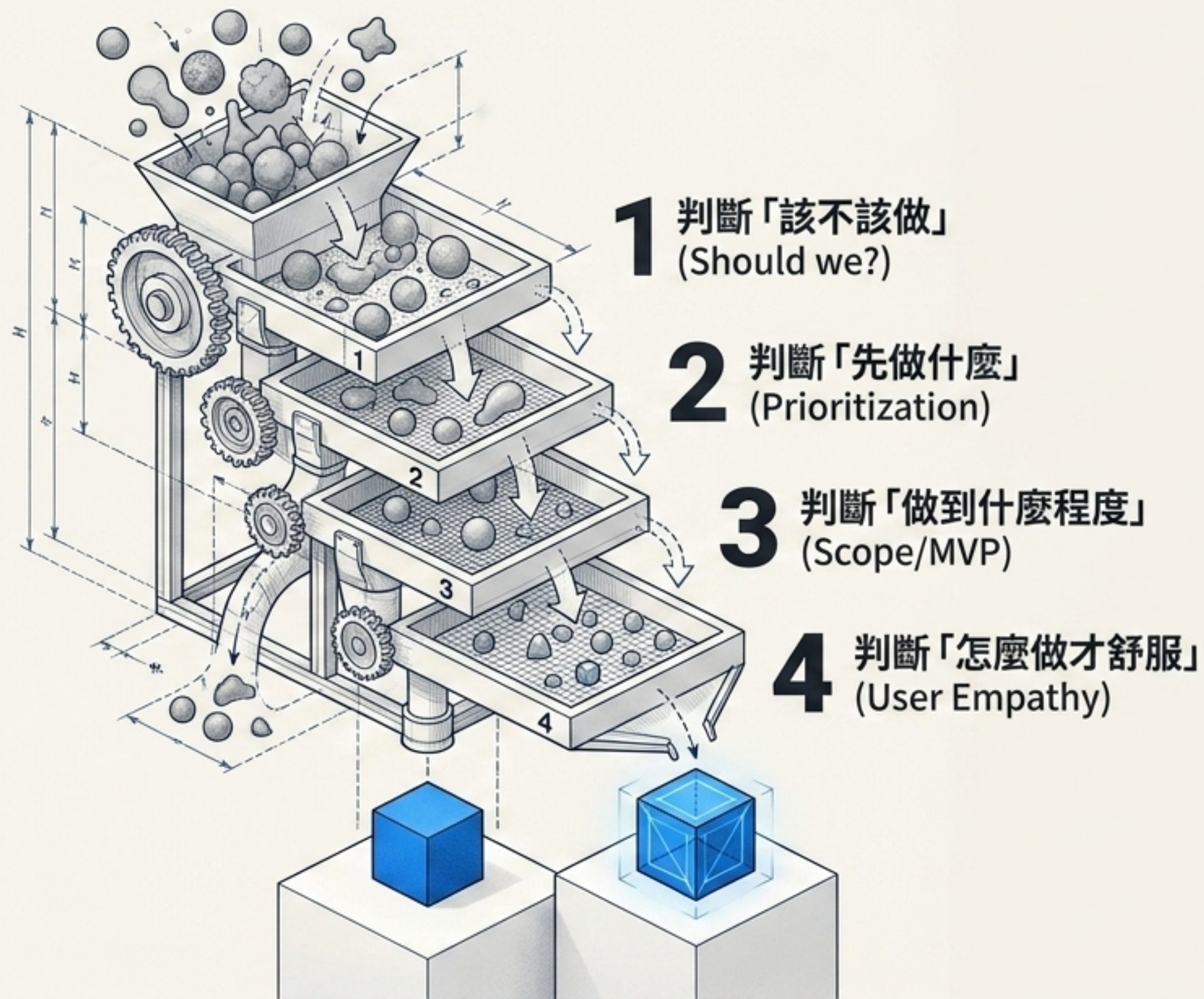
AI 讓「寫程式」變便宜，但沒有讓「判斷什麼值得做」變便宜。

重新定義產品感 (Product Sense)



在眾多可能性裡，判斷哪一件事最值得做，是未來最稀缺的能力。

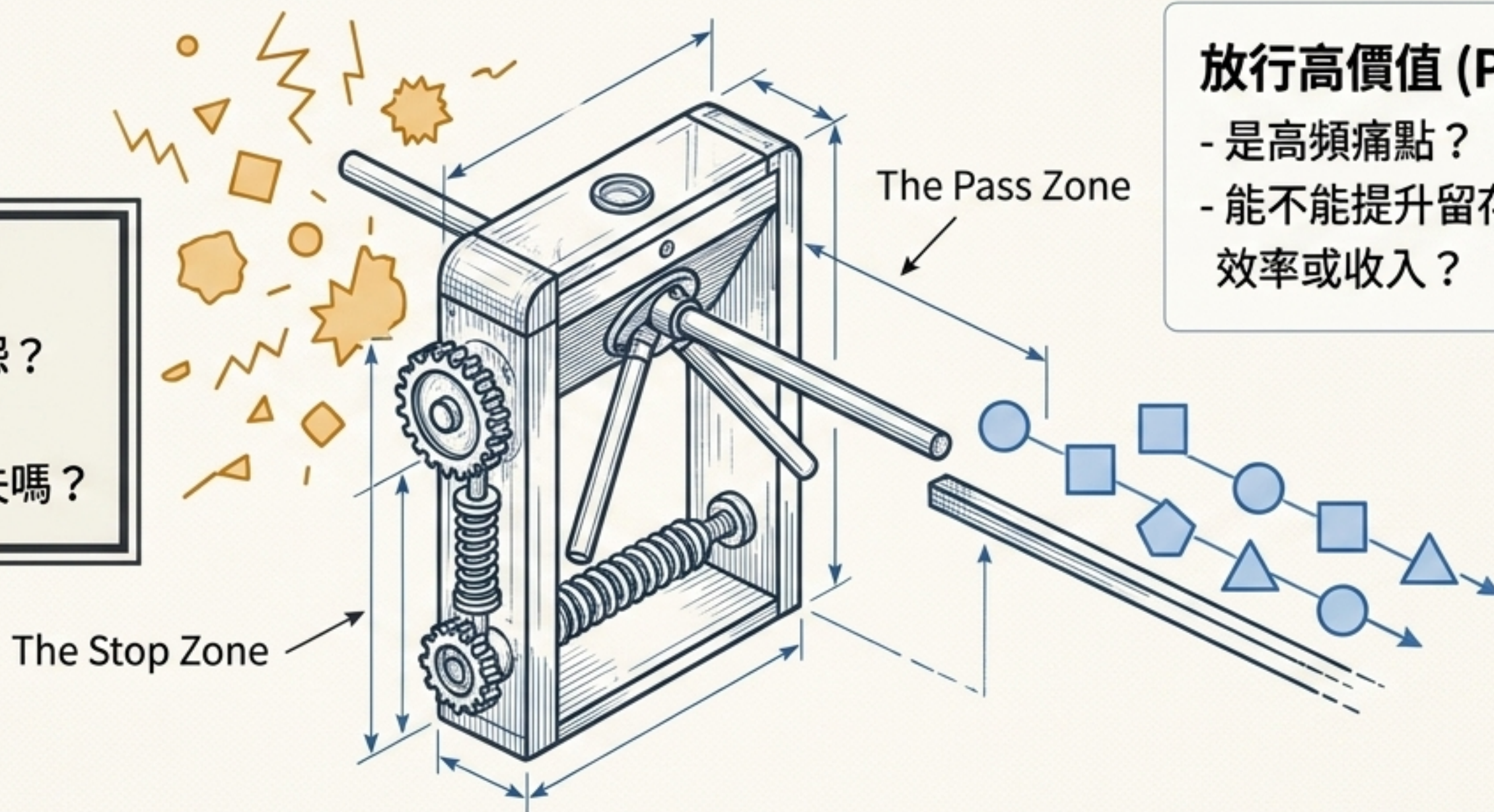
產品感的四層決策流



第一層：懂得拒絕不重要的需求

擋下雜訊 (Stop)：

- 是單一使用者的抱怨？
- 老闆突發奇想？
- 不解會造成明顯損失嗎？

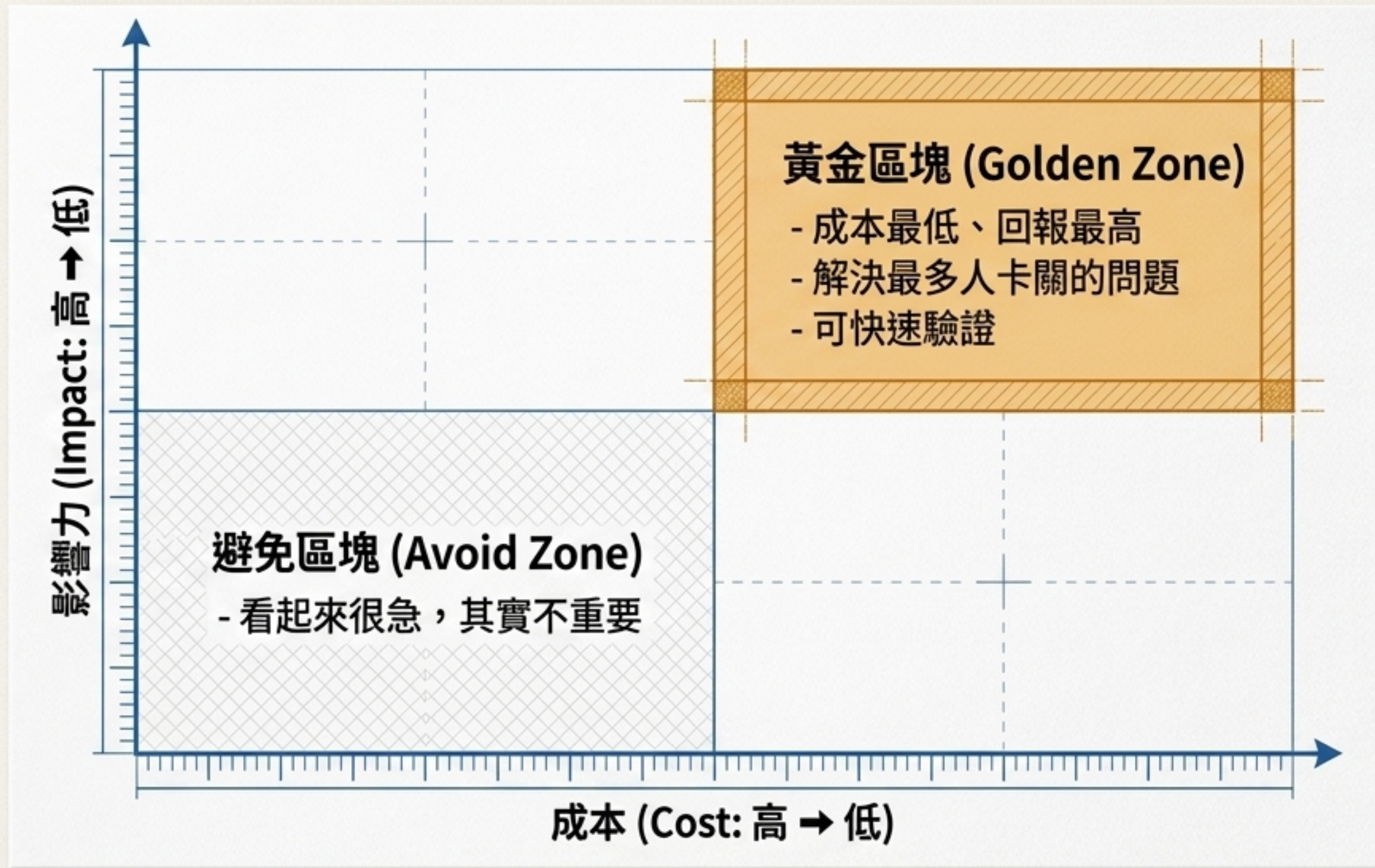


放行高價值 (Pass)：

- 是高頻痛點？
- 能不能提升留存、轉換、效率或收入？

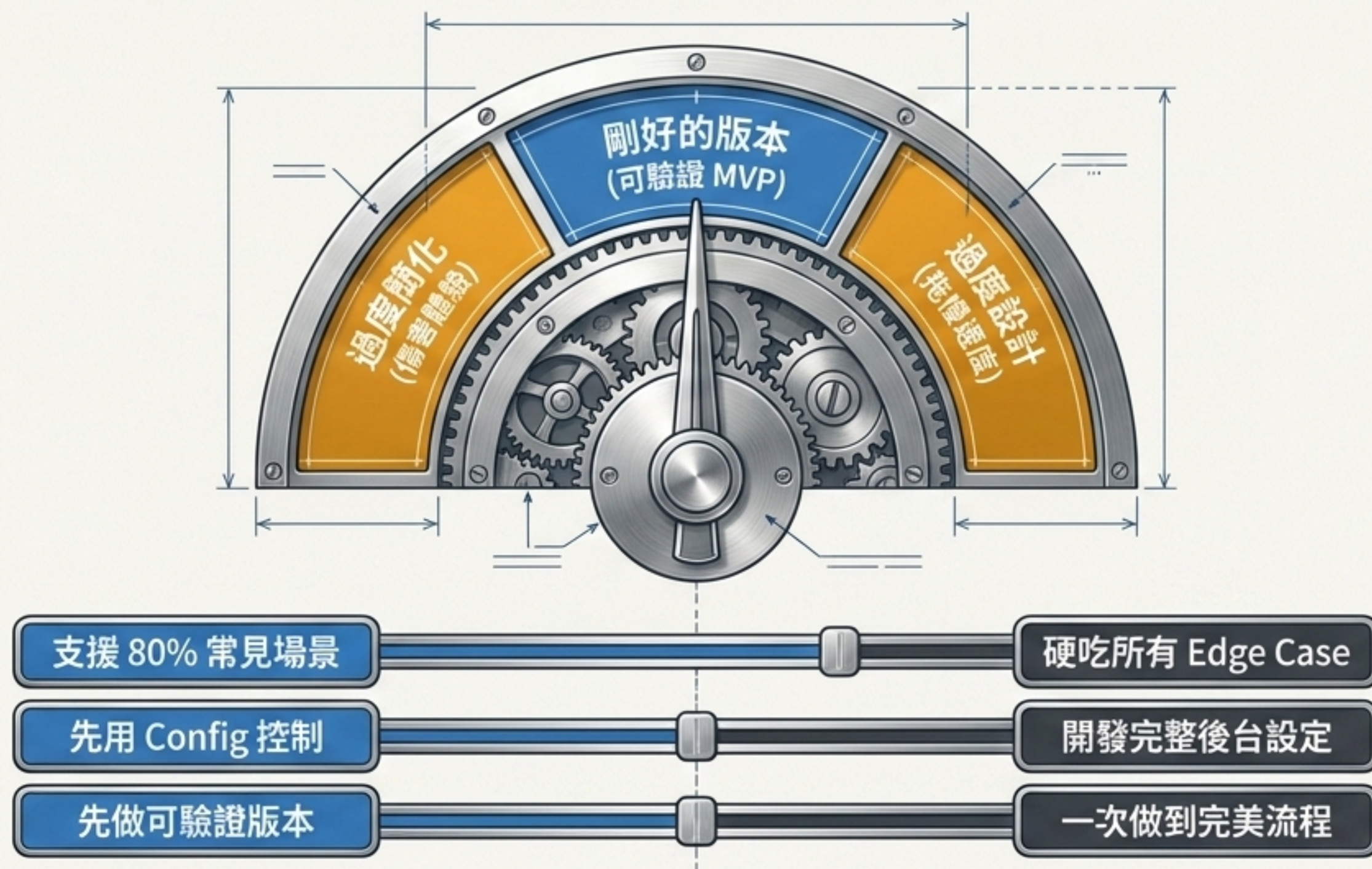
不是使用者有反饋就一定要做，也不是老闆想到就立刻做。產品感的第一層，是擋下雜訊。

第二層：不再問「能不能做」，而是問「先做什麼」



產品感不是把所有需求排進 Roadmap，而是精準抓出「卡住最多使用者的瓶頸」優先擊破。

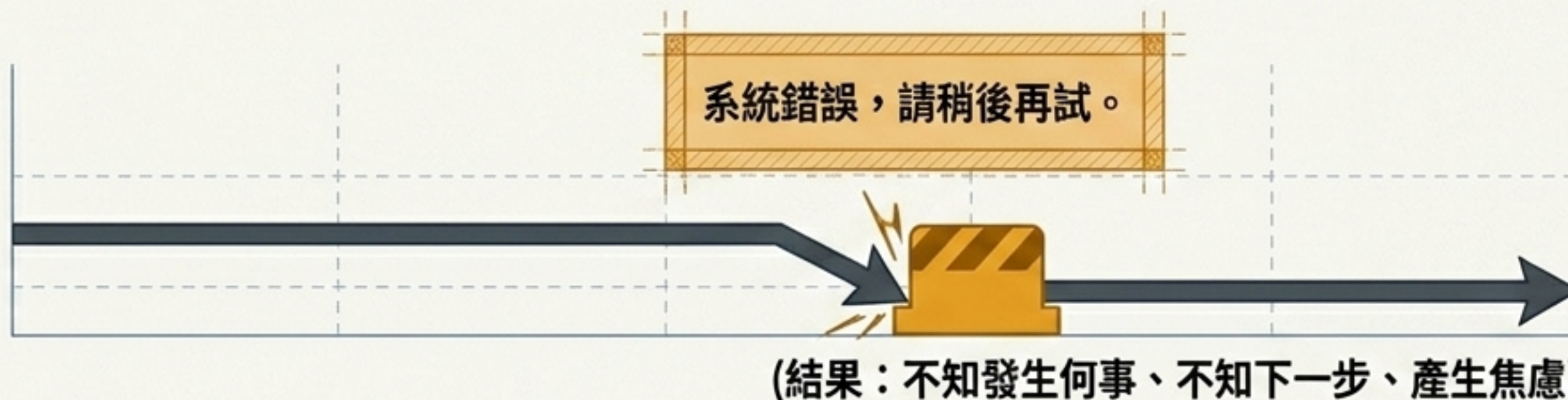
第三層：抓到「剛好」的交付程度



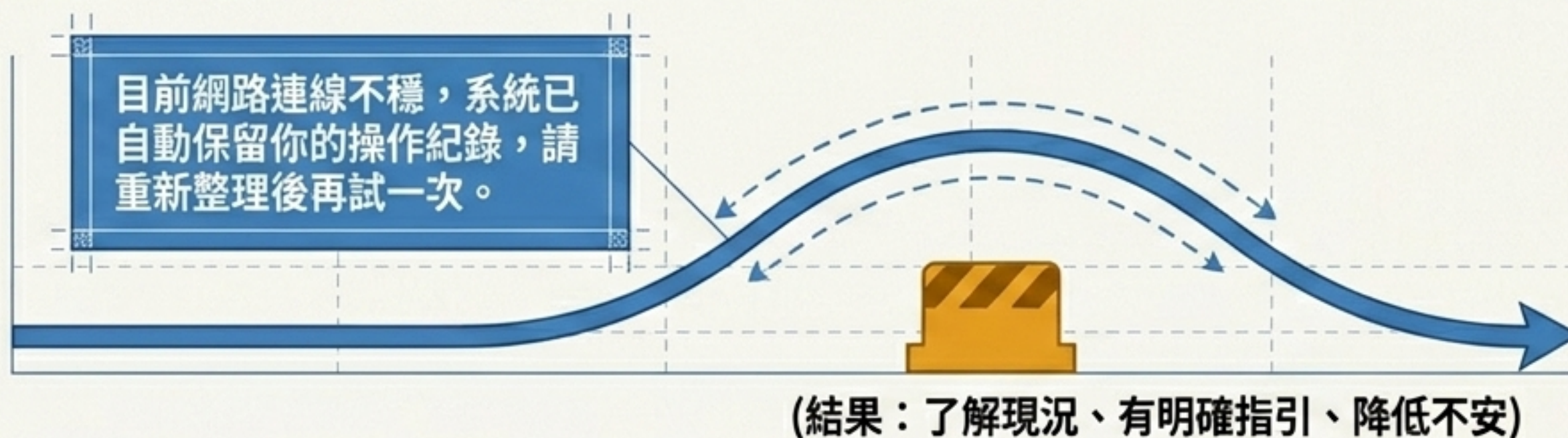
很多功能不是「做或不做」的二分法，而是裁量出不會傷害體驗、又不會拖慢速度的最佳切入點。

第四層：站在當下情境思考「怎麼做才舒服」

沒產品感
(Without Product Sense)



有產品感
(With Product Sense)



產品感也包含使用者體驗。核心是讓使用者「順、快、安心」。

角色進化 (Eng)：工程師的價值公式



工程師的產品感

= [技術成本判斷] × [使用者價值判斷]



思維轉換

- ➡ 舊思維：PM 說什麼，我就做什麼。
- ✅ 新思維：利用對實作成本的敏銳度，幫團隊避開災難。

決策指南矩陣

- | | |
|-----------------|----------------|
| ➡ 1 小時能解痛點 | ➡ 直接做 |
| ⚠️ 看似簡單但會導致維護災難 | ➡ 提出低成本替代方案 |
| ⏸️ 要動資料模型/改大架構 | ⏸️ 暫停，回頭確認商業價值 |

好工程師不只完成需求，更是幫團隊避免「做錯東西」的最後防線。

角色進化 (PM)：從「轉交窗口」到「決策中樞」

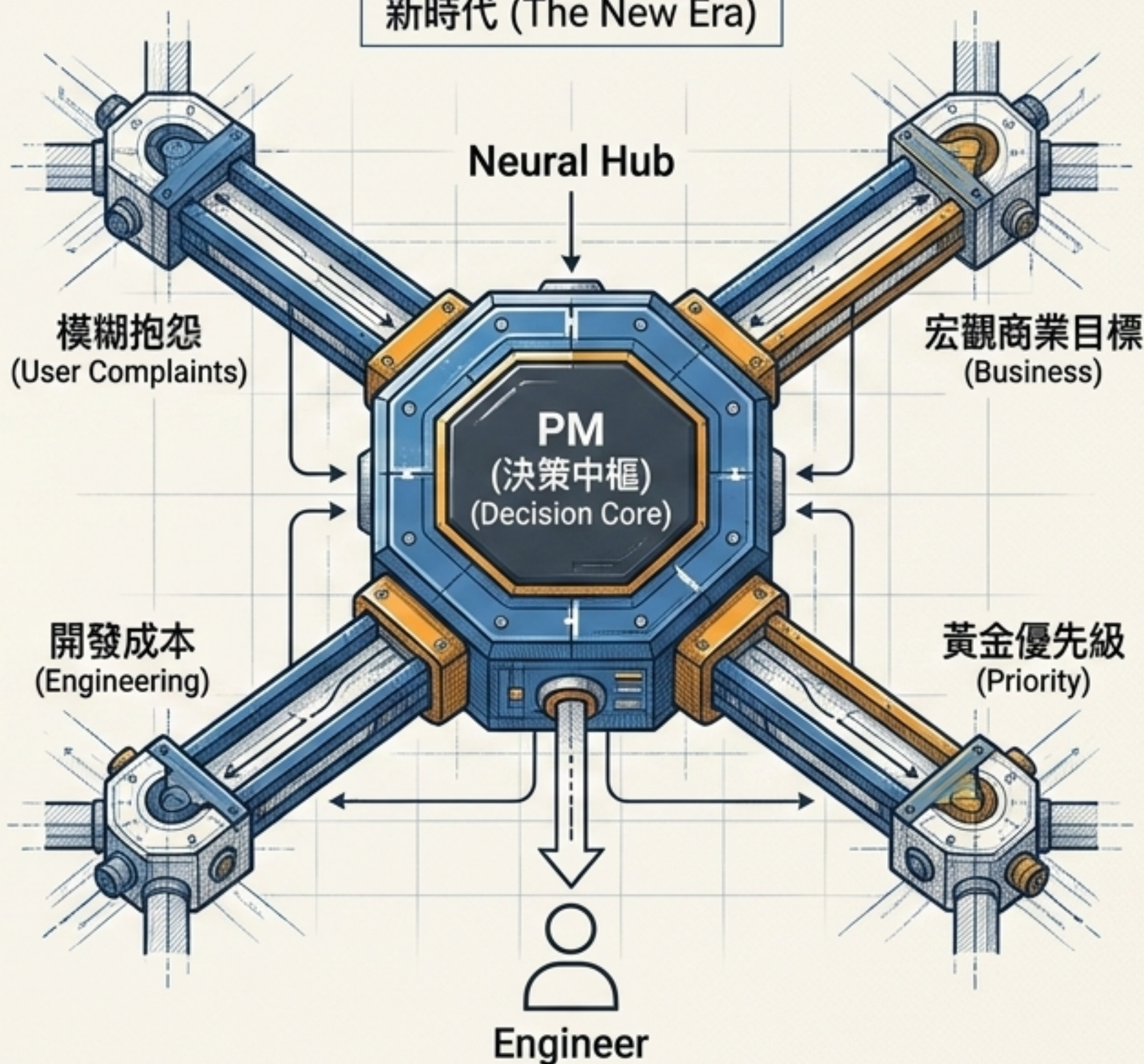
舊時代 (The Old Era)



AI 時代 PM 的核心價值

- ✓ 把模糊的問題 → 變清楚
- ✓ 把使用者的抱怨 → 轉成真正的需求
- ✓ 把宏觀商業目標 → 轉為落地產品策略
- ✓ 把龐大功能 → 定義成可交付、可驗證的版本

新時代 (The New Era)



核心檢驗：思維對比矩陣

沒產品感		有產品感
[收到需求]	使用者說要就排進去	先判斷背後問題與影響範圍
[排優先級]	誰喊得大聲先做誰	綜合看價值、成本、風險與時機
[定義功能]	一次做完整規格	先做最小可驗證版本 (MVP)
[看工程端]	只問什麼時候完成	一起判斷成本與低成本替代方案
[看體驗結果]	上線就結束，有做就好	要求順、快、安心，上線看數據修正

決策儀表板：產品感強的人都在問什麼？

需求診斷

這個需求背後的真正問題？不解的損失是什麼？
高頻還是特例？

優先級診斷

哪個成本最低、收益最高？
哪些聲音大但價值低
可以先不做？

體驗診斷

使用者卡在哪？流程是否直覺？
有沒有多點、多想、多等？

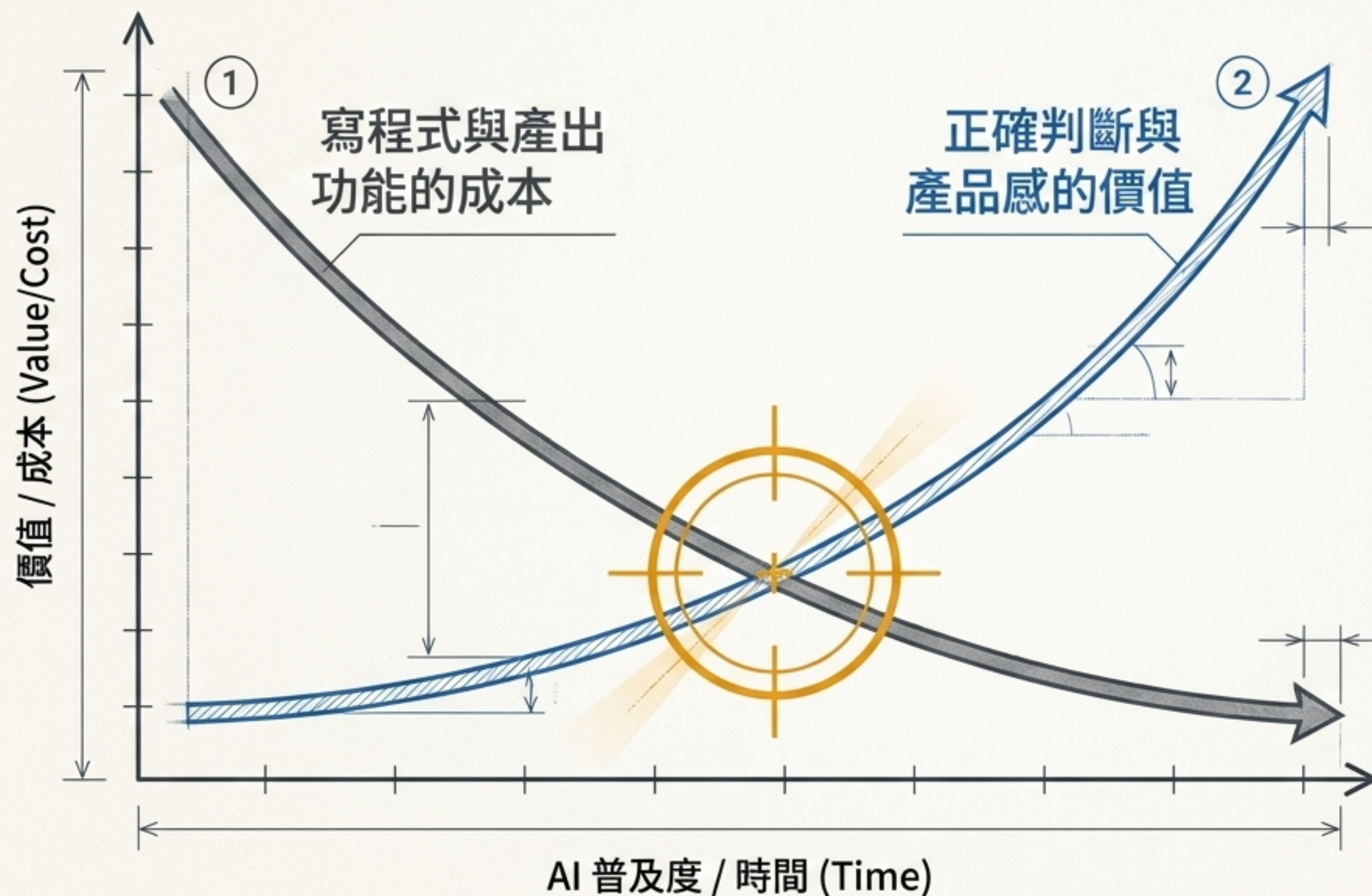
技術/成本診斷

實作成本多高？會造成架構負債嗎？
是否為了Edge Case 過度設計？

驗證診斷

做完看什麼指標？怎麼知道有效？
沒效果下一步怎麼調？

價值反轉：不要只追求產出速度，要追求正確產出

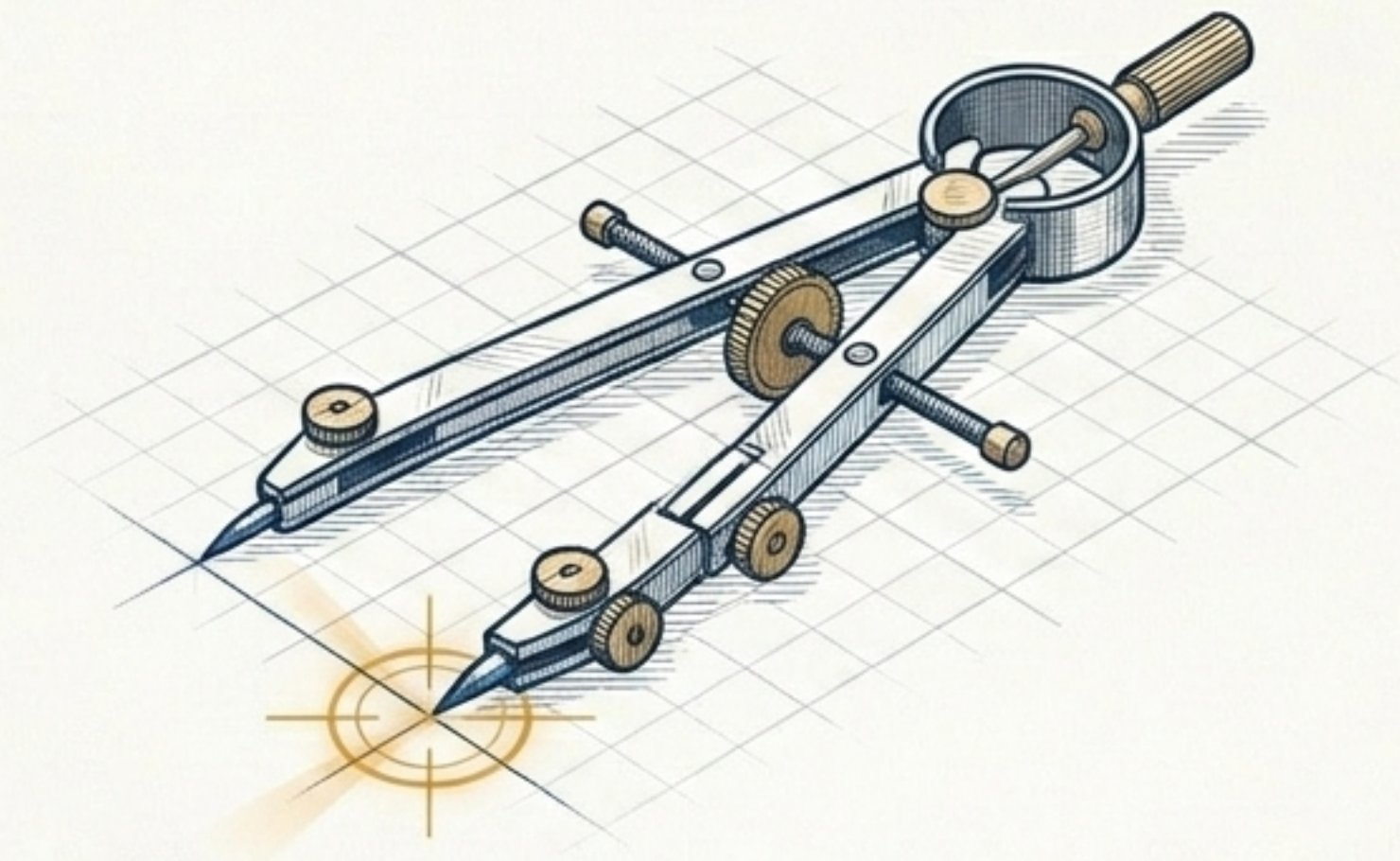


AI 會讓團隊「做得更快」，但不保證「做得更對」。

如果沒有產品感，速度越快，只是越快做出一堆沒價值的功能。

真正有效率的團隊，能把產能集中在最有價值的地方。

產品感，是未來的全方位必修課



產品感不是 PM 的專屬，也不是工程師不需要的軟技能。在 AI 時代，未來的競爭不只是誰寫得快，而是誰更知道：什麼該做、先做什麼、做到什麼程度。

寫程式會越來越便宜，但正確的判斷會越來越貴。
